

Natural Language Processing

Text Classification & POS Tagging with CNNs

Dec 2025

**Maria Schoinaki
Electra Papadopoulou
Leonidas Kontogiannis**

You can view the full code implementation for the **text classification** in the following link:



You can view the full code implementation for the **POS tagger** in the following link:



All members have contributed equally in all tasks, markdown and report writing, synchronously throughout the assignment.

Tweet Sentiment Classification with CNNs

For this assignment we used the Twitter Sentiment Analysis dataset from Kaggle. This dataset consists of ~100.000 tweets (sentiment texts), the id of each tweet, and the sentiment (0 or 1), and it was used as the basis for training (70%), development (15%), and testing (15%) of the sentiment classifiers. The same dataset had been used for previous assignments, thus we will be brief in summarizing the preprocess and stage. For more details, you can read the report [here](#).

Data Preprocessing/ Cleaning

In order to preprocess the dataset, we implement **spaCy** and **regular expressions**.

- **Text Cleaning:**

For a dataset full of tweets, it is important to scrap the information that is not useful for sentiment classification, such as **symbol-prefixed words** (tagging, quotient, etc.) Non-word characters and extra spaces are also removed and lowercasing is applied.

- **Linguistic Normalization:**

To preprocess each tweet, we will utilize **POS** tagging, specifically spaCy's **English language model** (*en_core_web_sm*). Only **tokenization** and **lemmatization** were required.

The result of the process is a list of tweets.

Embedding Construction For the CNN

We will use the same learned **word embeddings** we used of [Bi-LSTM](#) model in our **CNN Model**. Word embeddings allow us to exploit syntactic and semantic structure and provide a more suitable input representation for recurrent neural networks.

Post pre-processing, we constructed a **vocabulary of 39,441 unique tokens**, augmented with two special tokens, `<pad>` and `<unk>`.

We trained skip-gram Word2Vec embeddings directly on the cleaned training corpus using an embedding size of 100, window size 5, min_count=1, and 10 epochs, resulting in vectors for all 39,441 unique tokens. Allowing the embedding layer to be fine-tuned during training empirically yielded better performance than pruning rare words. A small number of tweets became empty after preprocessing and were removed prior to Word2Vec training to avoid degenerate inputs. The learned embeddings were used to initialize a PyTorch `nn.Embedding` layer with trainable parameters, with special tokens initialized randomly. Tweets were then converted to indexed sequences and batched using a custom padding `collate_fn`, enabling efficient training of the CNN models on variable-length inputs.

Experimental Setup

For Exercise 1 we reuse exactly the same dataset, splits, and preprocessing pipeline as in Exercise 11 (Part 2) and Exercise 4 (Part 3):

- **Corpus:** twitter-sentiment-analysis2 (Kaggle), with ~100K tweets labeled with binary sentiment (0 = Negative, 1 = Positive).
- **Splits:** 70% train, 15% development, 15% test, stratified by label, using random_state=2025.
- **Preprocessing:** Text Cleaning and Linguistic Normalization, as described above.

Baseline Classifiers

We consider two baselines, as requested:

TF-IDF + Logistic Regression (best model from Exercise 11 part 2):

- Feature selection: SelectKBest(mutual_info_classif, k=30,000)
- Classifier: LogisticRegression with C=1 (best value found in Exercise 11)

We reuse this tuned **TF-IDF + LR** as our strong linear baseline and retrain it on the same train/dev/test split used for the MLP.

MLP with Truncated TF-IDF SVD vectors (From Exercise 4 in part 3):

- Input layer: 500-dimensional SVD features.
- Hidden layers: a single fully connected layer with:
 - hidden_sizes = [512]
 - Activation: ReLU
 - Dropout: 0.7

- Output layer: linear layer with 2 units (Negative, Positive), trained with cross-entropy loss.

BiLSTM with Word Embeddings (From Exercise 1 in part 4):

Recurrent layer:

- BiLSTM, hidden size = **128** (per direction).
- Output hidden states from both directions concatenated.
- **2** stacked Layers

Classifier head:

- Fully connected layer mapping BiLSTM outputs to class logits (Negative, Positive).
- Global average pooling over time used to obtain fixed-length sentence representations.

Additionally, we have included a **majority baseline** classifier in our comparisons that serves as a lower bound for performance. The implementation was via scikit-learn's Dummy Classifier and it was trained on labels only.

Model Architecture

Network architecture

The final best configuration (after tuning 10 random configurations) is:

Convolutional encoder (stacked CNN):

The model uses a **stack of 2 convolutional blocks**, each consisting of:

- **Input layer:** token indices mapped to pretrained embeddings (fine-tuned), embedding dropout 0.4.
- **Convolutional encoder (stacked CNN):**
 - Parallel 1-D convolution filters with **n-gram kernel sizes (2, 3)**
 - **64 output channels** per filter
 - ReLU activation
 - Concatenation of feature maps
 - A **projection layer** that maps the concatenated features back to the embedding dimension
 - **Residual connections** to preserve input information
 - Layer normalization for stable training
 - These stacked convolutional blocks allow the model to capture **local n-gram patterns** repeatedly at deeper hierarchical levels.
- **Global pooling layer:** the sequence representation is aggregated using **global max-pooling**
- **Classifier head:** linear layer reducing hidden dimension by half → ReLU → dropout 0.4 → final linear layer with 2 outputs (Negative, Positive), trained with cross-entropy loss.

Data loaders

Tweets are tokenized and converted to index sequences with **<pad>** and **<unk>** tokens. Variable-length sequences are padded to the batch maximum.

- Batch size: 256
- Training loader shuffled, dev/test loaders not shuffled

This allows efficient mini-batch training using packed sequences, maintaining proper handling of variable-length inputs.

Training

The model is trained using mini-batches of tokenized sentences. Each epoch performs a full pass over the training set, computing the forward pass, loss, and gradients, followed by an optimizer update. We use Adam with optional weight decay, cross-entropy loss, and gradient clipping to stabilize CNN training. After every epoch, the model is evaluated on both the training and development sets, computing accuracy, loss, and Macro-F1. Early stopping is applied based on development Macro-F1: if the score does not improve for several epochs (**patience = 5**), training stops automatically. The best model parameters are tracked and restored at the end, ensuring that evaluation always uses the best-performing checkpoint rather than the final epoch.

Hyperparameter Tuning (Random Search)

We perform **random search** over a structured hyperparameter space. Random search is chosen over exhaustive grid search because grid search becomes inefficient when multiple hyperparameters interact non-linearly. Random sampling explores a larger variety of configurations at a fraction of the computational cost.

Training Procedure

For each sampled configuration:

1. A new **StackedCNNClassifier** model is created using a random sample of hyperparameters.
2. The model is trained for up to **10 epochs** with **early stopping** (patience=5).
3. After training, the **Macro F1-score on the development set** is used as the selection metric.
4. The **best-performing model**, its hyperparameters, and its full training history are stored.

Best Model and Hyperparameters

After evaluating **10** random configurations, the best model was obtained with the following hyperparameters:

- kernel_sizes = 2 and 3 **n-gram**
- num_layers = 2
- dropout = 0.4
- out_channels_per_filter = 64
- learning rate = 0.01
- fine_tune_embeddings = True

The **best model's macro F1 score** on the development set was **~0.76**.

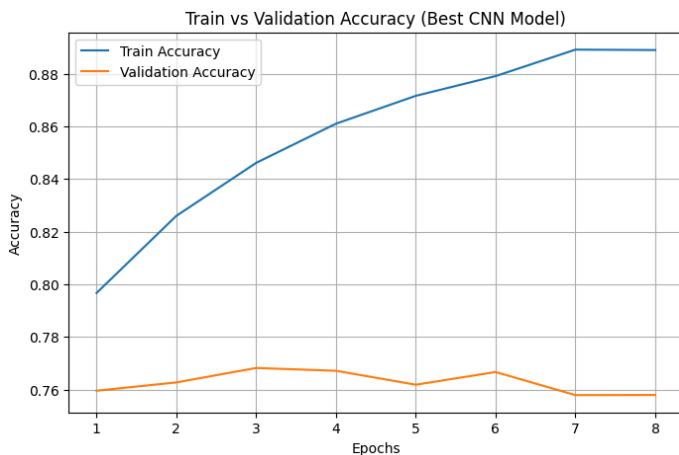
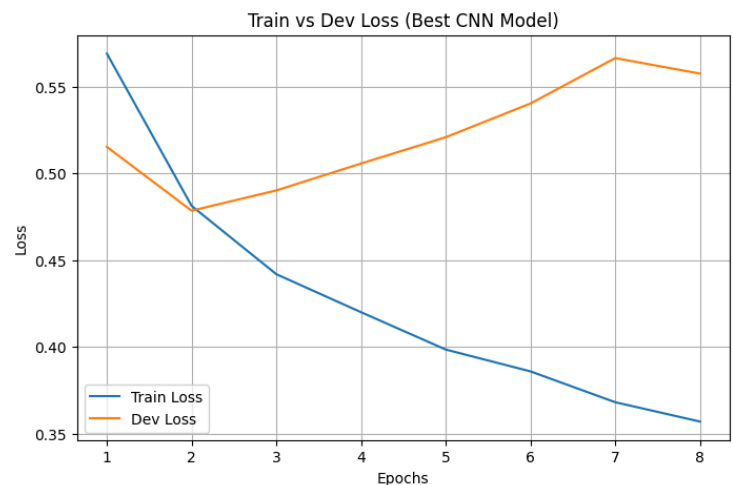
Notes on Implementation

- We ensured **no duplicate configurations** were sampled during the random search.
- The training loop returned a full **history** containing: `train_loss`, `dev_loss`, `train_accuracy`, `val_accuracy`, `train_f1`, and `val_f1` for each epoch, enabling detailed learning curve analysis.

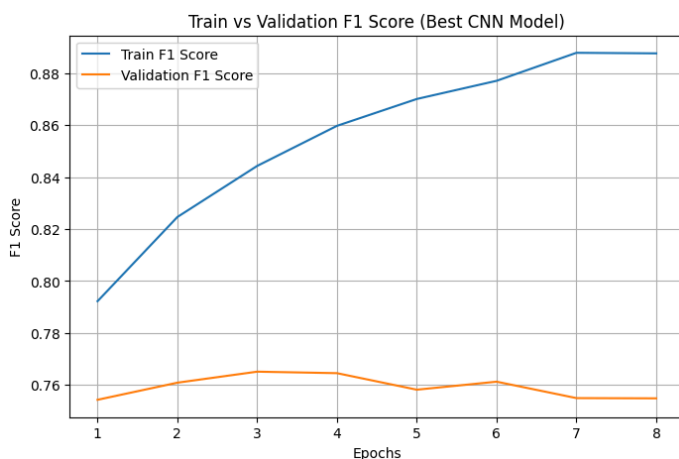
Loss Curves

After training, we analyze the behavior of the best model across epochs by visualizing the training history:

- **Train vs. Dev Loss:** The curves help determine whether the model continues learning (**training loss decreasing**), whether it generalizes well (**dev loss follows train loss without diverging**) and whether overfitting occurs (**dev loss increases while train loss continues to drop**).



- **Train vs. Validation Accuracy:** We want to check if the model is consistently improving its predictions.



- **Train vs. Validation F1 Score:** It monitors whether the model improves in its ability to handle all classes consistently, and whether precision-recall tradeoffs change over training, though it is more informative when classes have uneven representation.

Additionally, all of these curves can show when the model starts overfitting, which is happening in this case, even though we have tuned our model to the best epoch. The dev loss does not follow

the train loss, and thus the validation accuracy and F1-score do not improve that greatly over time.

Evaluation

We report per-class **Precision (P)**, **Recall (R)**, **F1**, and **PR-AUC** and we also include **Macro-Averaged P/R/F1/PR-AUC**. Models are **trained** on *train* dataset, **tuned** in *dev* dataset and **evaluated** in *test* dataset.

Systems Compared

- **Majority baseline** (*dummy classifier-most_frequent*): Always predicts the training set’s majority class.
- **LR (TF-IDF) tuned**: Logistic Regression with TF-IDF word n-grams (*uni, bi, tri*).
- **MLP (TF-IDF) tuned**: Multi-Layered Perceptron with SVD features.
- **BiLSTM** (word embedding) **tuned**: Bidirectional LSTM using pretrained word embeddings.
- **CNN** (word embedding) **tuned**: Stacked CNN using pretrained word embeddings.

Results

We provide three tables (*train, dev, test*). Each table has one row per model and four columns per class (*P, R, F1, PR-AUC*), followed by *Macro-P, Macro-R, Macro-F1, Macro-PR-AUC*.

Training Set

=== TRAIN RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435364	0.564636	1.000000	0.721747	0.564636	0.282318	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.798243	0.832010	0.814777	0.882391	0.866103	0.837854	0.851745	0.933030	0.832173	0.834932	0.833261	0.907710
2	MLP	0.833710	0.678689	0.748254	0.862213	0.783318	0.895622	0.835714	0.907433	0.808514	0.787156	0.791984	0.884823
3	BiLSTM	0.826609	0.823558	0.825081	0.895426	0.864351	0.866811	0.865579	0.939348	0.845480	0.845185	0.845330	0.917387
4	CNN	0.807150	0.849489	0.827779	0.893612	0.879067	0.843516	0.860925	0.940486	0.843109	0.846503	0.844352	0.917049

- The **majority baseline** reaches Macro-F1 ≈ 0.36 and Macro PR-AUC ≈ 0.50 (*constant prediction yields poor F1 but PR-AUC near the class prior*).
- The **CNN** and **BiLSTM** model achieves a high training score that surpasses our MLP and LR which it is also expected, as these models are more **word-order aware**.

Development Set

=== DEV RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435391	0.564609	1.000000	0.721725	0.564609	0.282304	0.500000	0.360863	0.500000
1	TF-IDF+LR	0.733579	0.762787	0.747898	0.814300	0.811282	0.786372	0.798633	0.881143	0.772430	0.774580	0.773265	0.847721
2	MLP	0.711877	0.581930	0.640377	0.740417	0.717391	0.818375	0.764563	0.820410	0.714634	0.700152	0.702470	0.780414
3	BiLSTM	0.752113	0.739157	0.745579	0.823291	0.801382	0.812033	0.806672	0.889064	0.776747	0.775595	0.776125	0.856177
4	CNN	0.726097	0.751000	0.738338	0.807238	0.802657	0.781417	0.791895	0.877310	0.764377	0.766208	0.765116	0.842274

- **Baseline:** Macro F1≈0.3609, Macro PR-AUC=0.5000
- **LR (TF-IDF):** Macro F1=0.773, PR-AUC=0.847
- **MLP (TF-IDF):** Macro F1=0.702, Macro PR-AUC=0.780
- **BiLSTM(world embeddings):** Macro F1=0.776, Macro PR-AUC=0.856
- **CNN(world embeddings):** Macro F1=0.765, Macro PR-AUC=0.842

Observation: CNN is a little bit worse than BiLSTM, and clearly dominates the MLP.
The baseline is far behind, as expected.

Test Set

=== TEST RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435362	0.564638	1.000000	0.721749	0.564638	0.282319	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.732353	0.754671	0.743344	0.817290	0.806288	0.787342	0.796702	0.879971	0.769320	0.771006	0.770023	0.848630
2	MLP	0.712849	0.575191	0.636664	0.746861	0.714902	0.821348	0.764438	0.820577	0.713876	0.698270	0.700551	0.783719
3	BiLSTM	0.755556	0.732759	0.743982	0.824325	0.798888	0.817449	0.808062	0.884238	0.777222	0.775104	0.776022	0.854281
4	CNN	0.728339	0.745382	0.736762	0.805632	0.800338	0.785917	0.793062	0.872382	0.764339	0.765650	0.764912	0.839007

- **Baseline:** Very poor for class 0 (F1=0), okay for class 1; overall macro F1 is low (0.36).
- **TF-IDF + LR:** Solid balanced performance, macro F1 ~0.77, strong precision and recall for both classes and PR-AUC ~ 0.848
- **MLP:** Worse than **TF-IDF+LR** and **BiLSTM**, macro F1 ~0.70.
- **BiLSTM:** Close to the **TF-IDF + LR**, balanced precision and recall across both classes; macro F1 ~0.776, slightly worse than TF-IDF+LR, and PR-AUC ~0.854 slightly better.
- **CNN:** Close to the **BiLSTM** and **TF-IDF + LR**, balanced precision and recall across both classes; macro F1 ~0.764, slightly worse than TF-IDF+LR, and PR-AUC ~0.839 slightly better.

Character-Aware CNN and Temporal Averaging

In addition to the word-level CNN classifier, we implemented a character-aware CNN that incorporates subword information into word representations. Alongside pretrained word embeddings, the model learns

character embeddings that are processed by a character-level CNN, whose output is concatenated with the corresponding word embedding before being fed into the stacked word-level CNN. This extension aims to improve performance to informal spellings, rare words, and out-of-vocabulary tokens, which are common in tweets.

We also experimented with temporal weight averaging (CNN-TA) to improve training stability. During training, we maintain a running average of the model parameters across epochs and evaluate the temporally averaged model after training. This approach is intended to smooth parameter updates and reduce variance without changing the underlying architecture.

Our results from this are as follows:

=== TRAIN RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	CNN	0.787948	0.826787	0.806901	0.879636	0.861179	0.828452	0.844499	0.930167	0.824564	0.827620	0.825700	0.904902
1	CNN-char	0.857039	0.738550	0.793395	0.894945	0.817842	0.905017	0.859224	0.936913	0.837440	0.821784	0.826310	0.915929
2	CNN-TA	0.797577	0.757002	0.776760	0.862175	0.819722	0.851873	0.835488	0.919323	0.808649	0.804438	0.806124	0.890749
=== DEV RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	CNN	0.719783	0.753768	0.736383	0.803364	0.802833	0.773585	0.787937	0.874687	0.761308	0.763676	0.762160	0.839025
1	CNN-char	0.760919	0.621655	0.684273	0.790795	0.744203	0.849294	0.793283	0.862064	0.752561	0.735474	0.738778	0.826430
2	CNN-TA	0.729698	0.677176	0.702457	0.781668	0.764025	0.806455	0.784667	0.860821	0.746862	0.741816	0.743562	0.821245
=== TEST RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	CNN	0.720749	0.740610	0.730544	0.805953	0.795931	0.779042	0.787396	0.872773	0.758340	0.759826	0.758970	0.839363
1	CNN-char	0.763133	0.623922	0.686542	0.791058	0.746076	0.850877	0.795038	0.862629	0.754605	0.737400	0.740790	0.826844
2	CNN-TA	0.728737	0.668719	0.697439	0.780500	0.760116	0.808321	0.783478	0.856789	0.744426	0.738520	0.740459	0.818645

Training set: The char-aware CNN achieves the highest Macro-F1 and Macro-PR-AUC among the CNN variants, followed by the word-only CNN and the temporally averaged CNN. This suggests that character-level information improves the model’s capacity to fit the training data.

Development set: The word-only CNN performs best in terms of Macro-F1 and Macro-PR-AUC, with CNN-TA and CNN+char yielding slightly lower but comparable results. The character-aware model shows higher recall for the positive class, while the temporally averaged CNN produces more balanced precision–recall trade-offs.

Test set: The word-only CNN slightly outperforms the other variants, with CNN+char and CNN-TA performing very similarly. While character-level modeling improves recall for sentiment-bearing variations in tweets, these gains do not consistently translate into higher macro-averaged performance.

POS Tagging

For this assignment we used the UD Ancient Greek PROIEL treebank from Universal Dependencies. The corpus contains morphologically and syntactically annotated **Ancient Greek** texts. After loading the dataset, we obtained **15,016** sentences for training, **1,019** sentences for development, and **1,047** sentences for testing. Each sentence consists of a sequence of tokens paired with their corresponding Universal POS tags (UPOS). The same dataset had been used for previous assignments, thus we will be brief in summarizing the preprocess and stage. For more details, you can read the report [here](#).

Data Preprocessing

For the CNN model, sentences are processed as padded sequences: local context is captured implicitly by convolutional kernels of varying widths. At sentence boundaries, missing context is filled with the special <PAD> token.

The following table summarizes **key statistics** for the training, development, and test subsets of the **Ancient Greek PROIEL** treebank:

```
TRAIN stats:
sentences:      15016
tokens:         187039
avg sent length: 12.46
median sent len: 10.00
vocab size:     30349
number of tags: 13
most common tags:
VERB: 34517
NOUN: 31424
DET: 28070
ADV: 20288
PRON: 19756
ADP: 13812
CCONJ: 12117
ADJ: 10361
PROPN: 7745
SCONJ: 3610
```

```
DEV stats:
sentences:      1019
tokens:         13652
avg sent length: 13.40
median sent len: 11.00
vocab size:     4541
number of tags: 13
most common tags:
VERB: 2671
NOUN: 2185
DET: 2082
PRON: 1446
ADV: 1378
CCONJ: 978
ADP: 922
ADJ: 729
PROPN: 618
SCONJ: 257
```

```
TEST stats:
sentences:      1047
tokens:         13314
avg sent length: 12.72
median sent len: 11.00
vocab size:     4495
number of tags: 13
most common tags:
VERB: 2494
NOUN: 2209
DET: 2006
PRON: 1427
ADV: 1416
CCONJ: 1011
ADP: 885
ADJ: 752
PROPN: 494
AUX: 248
```

We trained **Word2Vec** embeddings directly on the **training corpus**. Only the word sequences were used as input for Word2Vec, and we employed a **skip-gram architecture** with a vector dimension of **100**, a window size of 5, and a minimum frequency threshold of one. The Word2Vec model was trained for ten epochs. This produced distributional word vectors for all words seen in the training data.

Our Model

The POS tagger is a **sequence-based convolutional neural network (CNN)** that processes entire sentences using stacked residual convolutional blocks rather than recurrent units. Each word is first mapped to a continuous vector via an embedding layer initialized with pretrained Word2Vec embeddings,

which are fine-tuned during training. When enabled, a character-level CNN encodes each word as a sequence of characters using multiple convolutional filters and max-pooling, and this representation is concatenated with the word embedding to incorporate subword information. The combined representations are projected into a fixed hidden dimension and passed through multiple residual CNN blocks that apply parallel temporal convolutions, allowing the model to capture local contextual patterns while preserving sequence length. A final dropout layer and linear classifier predict a POS tag for each word position.

Training uses mini-batches of padded sentences and is optimized with Adam and cross-entropy loss while ignoring padded tokens. Model performance is evaluated after each epoch using development-set loss and macro-F1, with early stopping applied based on dev loss. The parameters corresponding to the best dev macro-F1 score are restored at the end of training to ensure stable generalization.

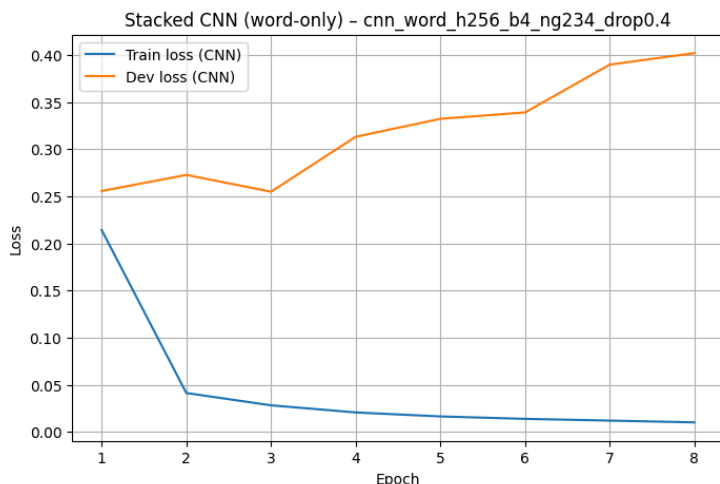
Fine Tuning and Training

To identify the best convolutional sequence model, we conducted a hyperparameter search over several stacked CNN configurations, varying the hidden dimensionality, number of residual convolutional blocks, kernel sizes, dropout rate, and learning rate. Each configuration was trained on the training set and evaluated on the development set using macro-F1 as the model-selection and early-stopping criterion. For every run, we tracked both training and development loss curves to assess convergence and generalization behavior. After training all candidates independently, the configuration achieving the highest development macro-F1 was selected as the final model.

Our best-performing word-only CNN uses four stacked residual convolutional blocks with a hidden dimension of 256 and parallel temporal kernels of sizes (2, 3, 4). A dropout rate of 0.4 was applied throughout the network, and optimization was performed using Adam with a learning rate of $7e-4$. This configuration achieved a development macro-F1 of 0.9346 and was retained as the final word-level CNN baseline.

We train the **final CNN** with:

- Hidden sizes: [256]
- Number of blocks: 4
- Kernel sizes: (2, 3, 4)
- Dropout: best value from dropout tuning (0.4)
- Epochs: best value from epoch tuning (20 in our run)
- Optimizer: Adam(lr= $7e-4$)



The training loss decreases monotonically, while the development loss initially decreases and then rises again, indicating overfitting if we were to train for too many epochs. However, by selecting the epoch with the best development macro-F1, the final model corresponds to an early epoch before severe overfitting sets in.

Character-Aware CNN Model

In addition to the word-level CNN, we implemented a character-aware CNN that augments each word representation with subword information. As in the word-only model, pretrained Word2Vec embeddings are used as input and fine-tuned during training. To incorporate character-level information, each word is first represented as a sequence of characters and passed through a character-level convolutional encoder with multiple kernel sizes and max-pooling. The resulting character-based word embedding is then concatenated with the corresponding word embedding and projected into the shared hidden space before being processed by the stacked residual CNN blocks. This allows the model to capture morphological patterns and orthographic features while retaining strong contextual representations.

Training, evaluation, and hyperparameter tuning followed the same procedure as for the word-only CNN, with additional tuning over character embedding size, number of character convolutional filters, and character kernel widths. Our motivation for this architecture is that character-level convolutions enable the model to handle rare, misspelled, and unseen words more effectively by exploiting subword structure, without relying solely on word-level embeddings.

CNN with Temporal Averaging

To improve training stability and reduce variance across epochs, we also implemented **temporal weight averaging**. Starting from the best-performing CNN+character configuration, the model was trained as before while simultaneously maintaining a running average of the model parameters across all training

epochs. At each update, the current parameter values were incorporated into the averaged weights, producing a smoothed version of the training trajectory. After training, we evaluated both the best single checkpoint selected by development macro-F1 and the temporally averaged model on the development set. As with the other CNN variants, we tracked training and development loss curves and compared macro-F1 scores to assess whether temporal averaging improved generalization.

Evaluation

Once again, we create three tables that summarize the training, development, and test set performance of all POS-tagging models across all UPOS categories. The results that interest us are as follows.

- On the **train set**, **CNN-TA > CNN-char > CNN > BiLSTM $\approx$ BiLSTM-TA > MLP > Baseline** in terms of Macro-F1 and Macro-PR-AUC.
- On the **dev set**, **CNN-TA $\approx$ CNN-char > BiLSTM-char > BiLSTM $\approx$ CNN > MLP > Baseline**. The per-word baseline still performs reasonably on frequent tags but struggles with rarer categories. The addition of character-level representations improves generalization for both CNN and BiLSTM architectures, while temporal averaging yields a small but consistent gain for the CNN.
- On the **test set**, CNN-TA and CNN-char slightly outperform the word-only CNN and remain competitive with BiLSTM-based models. While BiLSTM-char retains a small advantage overall, CNN-based sequence models achieve comparable Macro-F1 scores and strong PR-AUC values across most tags. The MLP remains competitive but consistently below sequence-based models, and the baseline performs worst on infrequent tags.

Overall, sequence models with CNNs, especially when incorporating character-level information or temporal averaging, achieve strong generalization across all UPOS tags.

--- TRAIN RESULTS ---																			
	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R
0	Baseline	0.9845	0.9908	0.9876	0.9759	0.9765	0.9940	0.9852	0.9711	0.9726	...	0.8797	0.7775	0.9996	0.9992	0.9994	0.9989	0.9671	0.9665
1	MLP (best config)	0.9954	0.9958	0.9956	0.9998	0.9980	0.9985	0.9983	1.0000	0.9943	...	0.9863	0.9986	0.9999	0.9997	0.9998	1.0000	0.9950	0.9946
2	BiLSTM	0.9990	0.9955	0.9972	1.0000	0.9991	0.9996	0.9993	1.0000	0.9978	...	0.9956	0.9998	0.9999	1.0000	1.0000	1.0000	0.9978	0.9978
3	BiLSTM-char	0.9946	0.9911	0.9928	0.9996	0.9942	0.9982	0.9962	0.9998	0.9922	...	0.9562	0.9894	0.9998	0.9997	0.9997	1.0000	0.9868	0.9896
4	BiLSTM-TA	0.9997	0.9999	0.9998	1.0000	0.9999	0.9999	0.9999	1.0000	1.0000	...	0.9996	1.0000	1.0000	1.0000	1.0000	1.0000	0.9997	0.9998
5	CNN	0.9910	0.9949	0.9929	0.9995	0.9940	0.9975	0.9957	0.9998	0.9858	...	0.9576	0.9867	0.9997	0.9998	0.9998	1.0000	0.9870	0.9875
6	CNN-char	0.9919	0.9925	0.9922	0.9996	0.9942	0.9981	0.9962	0.9999	0.9885	...	0.9629	0.9922	0.9998	0.9996	0.9997	1.0000	0.9880	0.9895
7	CNN-TA	0.9996	0.9990	0.9993	1.0000	0.9992	0.9999	0.9996	1.0000	0.9993	...	0.9970	1.0000	1.0000	1.0000	1.0000	1.0000	0.9987	0.9991

--- DEV RESULTS ---																			
	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R
0	Baseline	0.9774	0.7133	0.8247	0.7125	0.9683	0.9935	0.9807	0.9624	0.9556	...	0.8419	0.7201	0.7957	0.9989	0.8858	0.7950	0.9489	0.9039
1	MLP (best config)	0.9135	0.7243	0.8080	0.8915	0.9892	0.9902	0.9897	0.9993	0.9599	...	0.9213	0.9750	0.8636	0.9787	0.9175	0.9845	0.9490	0.9236
2	BiLSTM	0.8574	0.7668	0.8096	0.8969	0.9924	0.9892	0.9908	0.9993	0.9643	...	0.9261	0.9631	0.8477	0.9918	0.9141	0.9833	0.9478	0.9268
3	BiLSTM-char	0.9669	0.7202	0.8255	0.9062	0.9860	0.9935	0.9897	0.9991	0.9727	...	0.9198	0.9645	0.8643	0.9948	0.9250	0.9921	0.9551	0.9321
4	BiLSTM-TA	0.8576	0.7353	0.7917	0.8835	0.9839	0.9924	0.9881	0.9986	0.9613	...	0.8976	0.9591	0.8463	0.9809	0.9086	0.9815	0.9406	0.9207
5	CNN	0.9160	0.7325	0.8140	0.8794	0.9902	0.9913	0.9908	0.9991	0.9545	...	0.9091	0.9716	0.8465	0.9869	0.9113	0.9814	0.9528	0.9207
6	CNN-char	0.8814	0.7750	0.8248	0.9182	0.9871	0.9935	0.9903	0.9995	0.9636	...	0.9228	0.9748	0.9038	0.9880	0.9440	0.9924	0.9451	0.9374
7	CNN-TA	0.8691	0.7833	0.8240	0.9180	0.9860	0.9946	0.9903	0.9996	0.9679	...	0.9237	0.9720	0.8849	0.9933	0.9360	0.9948	0.9507	0.9364

=== TEST RESULTS ===

	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9730	0.7660	0.8571	0.7585	0.9723	0.9932	0.9827	0.9662	0.9746	...	0.8904	0.7953	0.7987	0.9976	0.8871	0.7972	0.9523	0.9197	0.9325	0.8811
1	MLP (best config)	0.9181	0.7899	0.8492	0.9206	0.9932	0.9955	0.9944	0.9998	0.9664	...	0.9431	0.9817	0.8626	0.9739	0.9149	0.9807	0.9513	0.9303	0.9397	0.9766
2	BiLSTM	0.8580	0.8032	0.8297	0.9124	0.9966	0.9955	0.9960	0.9996	0.9721	...	0.9457	0.9849	0.8515	0.9820	0.9121	0.9791	0.9498	0.9335	0.9405	0.9753
3	BiLSTM-char	0.9601	0.7673	0.8529	0.9230	0.9844	0.9966	0.9905	0.9998	0.9744	...	0.9214	0.9668	0.8615	0.9928	0.9225	0.9894	0.9546	0.9360	0.9435	0.9812
4	BiLSTM-TA	0.8951	0.8059	0.8481	0.9098	0.9888	0.9944	0.9915	0.9992	0.9620	...	0.9132	0.9746	0.8486	0.9796	0.9094	0.9780	0.9447	0.9330	0.9380	0.9736
5	CNN	0.9206	0.7859	0.8479	0.9083	0.9888	0.9955	0.9921	0.9996	0.9627	...	0.9259	0.9728	0.8476	0.9767	0.9076	0.9775	0.9507	0.9283	0.9382	0.9742
6	CNN-char	0.8623	0.8245	0.8430	0.9344	0.9910	0.9966	0.9938	0.9997	0.9721	...	0.9339	0.9816	0.9023	0.9848	0.9417	0.9910	0.9482	0.9443	0.9458	0.9834
7	CNN-TA	0.8796	0.8258	0.8519	0.9346	0.9855	0.9955	0.9904	0.9997	0.9720	...	0.9315	0.9757	0.8881	0.9932	0.9377	0.9936	0.9535	0.9438	0.9481	0.9834